Dr. Decode AI

MVP System Design & Build Blueprint

Version: 1.0 | **Project type:** AI-powered medical document explainer

> **Primary goal:** Build a secure, useful MVP with Python, FastAPI, PostgreSQL and Gemini, with no mandatory ongoing infrastructure cost.

1. Product definition

Dr. Decode AI helps people with little medical knowledge understand medical documents in plain English.

Supported documents

1. **Prescription** — explain written medication instructions without inventing dosages, frequency or duration.
2. **Laboratory test report** — explain results, units, reference ranges and findings.
3. **Medical report** — explain examination reports, discharge summaries and written radiology reports.

Supported formats: PDF, JPG, PNG and HEIC. One file per analysis, maximum 10 MB. Multipage PDFs are supported.

Explicitly out of scope for the MVP

- Interpreting raw X-ray, CT or MRI images.
- Chatting with the AI or asking follow-up questions.
- Saving analysis history.
- Downloading the original uploaded document from the app.
- Custom AI model training, multi-agent systems and complex background-job infrastructure.

Core principles

- Never invent facts that are absent from the document.
- Preserve original laboratory values, units and reference ranges.
- Separate facts extracted from the document from general educational suggestions.
- Flag unclear, missing or potentially concerning information.
- Never let the AI independently prescribe, change medication or invent a treatment plan.

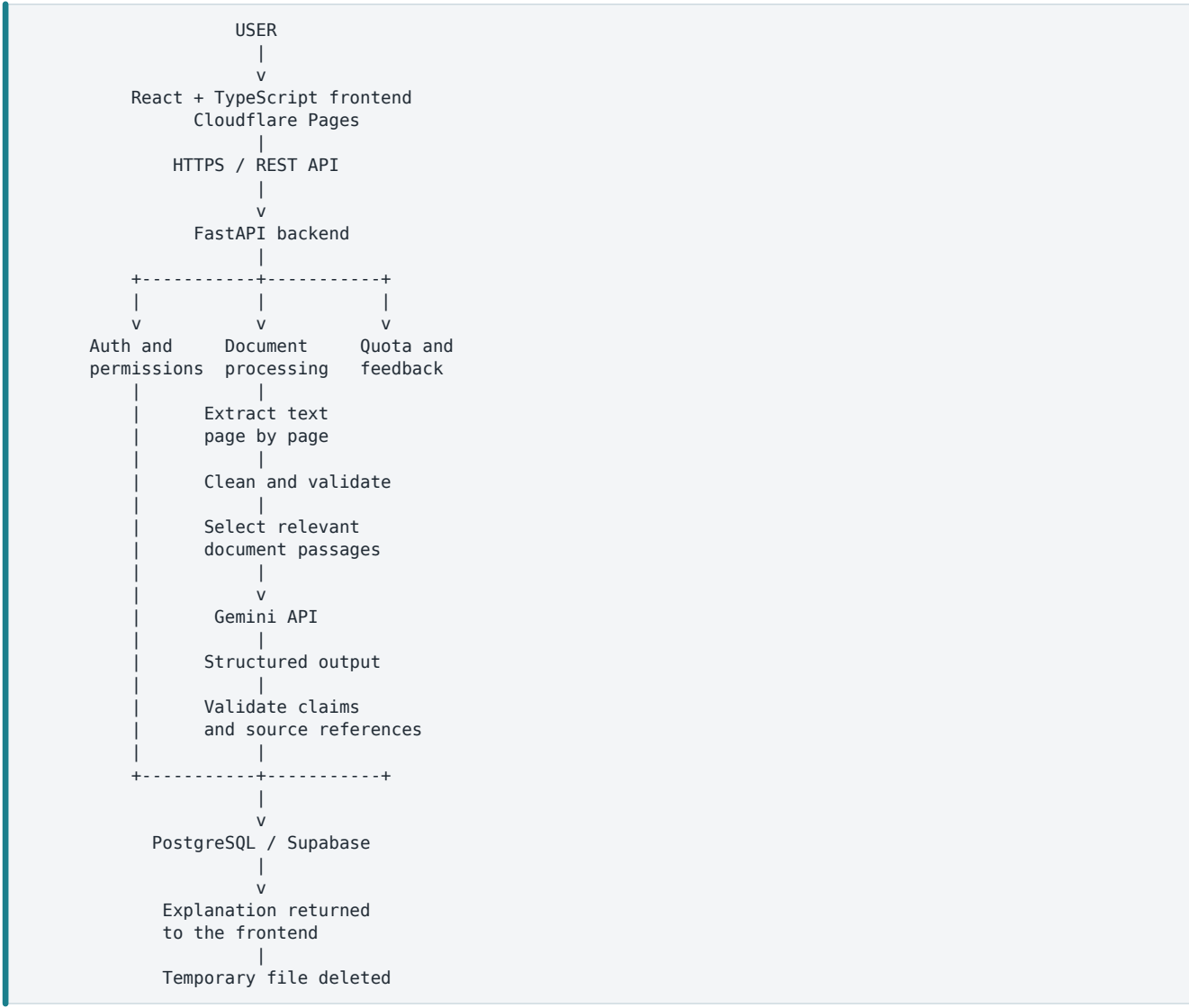
2. Technology stack

Component	Technology	Responsibility
Frontend	React + TypeScript + Vite	Upload interface and results
Frontend hosting	Cloudflare Pages	Serve the frontend
Backend	Python + FastAPI	API, validation and orchestration
Database access	SQLAlchemy	Database connections and queries
Database	PostgreSQL, preferably Supabase	Accounts-related data, quota and feedback
Authentication	Supabase Auth	Sign-in, verification and password reset
AI	Gemini API	Document explanation

Component	Technology	Responsibility
Document processing	PDF text extraction + image processing	Extract text and page references
Backend hosting	Undecided	Choose a suitable free-tier host at deployment
API testing	Postman	Test backend before frontend
Version control	Git + GitHub	Source code and project history

Cost constraint: We will target free tiers and avoid paid infrastructure dependencies. API quotas, SMTP limits, hosting sleep policies and provider terms must be checked before deployment. Zero cost cannot be guaranteed under every usage pattern.

3. High-level architecture



The diagram represents logical responsibilities, not necessarily separate deployed services. We can keep the backend modular while using a single FastAPI application.

Why this architecture?

The frontend handles presentation; the backend controls security, quota enforcement and AI calls. The browser must never receive a Gemini API key or direct database credentials.

This is **separation of concerns**: each component has a clear responsibility, making the application easier to maintain and extend.

4. Backend project structure

```
dr-decode-ai/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   ├── limits.py
│   │   │   └── logging.py
│   │   ├── api/
│   │   │   ├── dependencies.py
│   │   │   └── routes/
│   │   │       ├── health.py
│   │   │       ├── analysis.py
│   │   │       ├── quota.py
│   │   │       └── feedback.py
│   │   ├── database/
│   │   │   ├── session.py
│   │   │   └── schema.sql
│   │   ├── models/
│   │   │   ├── usage.py
│   │   │   └── feedback.py
│   │   ├── schemas/
│   │   │   ├── analysis.py
│   │   │   ├── prescription.py
│   │   │   ├── laboratory.py
│   │   │   └── medical_report.py
│   │   ├── services/
│   │   │   ├── document_service.py
│   │   │   ├── extraction_service.py
│   │   │   ├── retrieval_service.py
│   │   │   ├── gemini_service.py
│   │   │   ├── validation_service.py
│   │   │   ├── quota_service.py
│   │   │   └── pdf_service.py
│   │   └── utils/
│   │       ├── file_validation.py
│   │       └── cleanup.py
│   ├── requirements.txt
│   ├── .env.example
│   └── .gitignore
├── frontend/
│   └── (React + TypeScript app, built later)
├── README.md
└── .gitignore
```

Don't create every file on day one. Introduce modules as you implement their responsibilities.

> **Design rule:** API routes should coordinate work, not contain the entire business logic. Document extraction belongs in the extraction service; quota logic belongs in the quota service; Gemini calls belong in the Gemini service.

5. API design

All endpoints use the `/api/v1` prefix except the basic health endpoint.

Method	Endpoint	Purpose
GET	`/health`	Check whether the backend is running
GET	`/api/v1/quota`	Return attempts remaining and reset time
POST	`/api/v1/analysis`	Upload and analyze one document
POST	`/api/v1/feedback`	Submit a rating or problem report

Authentication is handled by Supabase Auth. The frontend sends its access token as a bearer token, and FastAPI verifies it before allowing protected operations.

Analysis request

`POST /api/v1/analysis`

Content type: `multipart/form-data`

Fields:

- `document_type`: `prescription`, `laboratory_report` or `medical_report`
- `file`: the uploaded document

The backend validates the token, category, actual file type, size and document content. It then checks quota, extracts the document, calls Gemini, validates the response and returns the structured explanation.

The response is JSON, not HTML. That allows the frontend to display the same data in the browser or turn it into a downloadable explanation PDF.

Error responses

Use consistent error codes and readable messages.

HTTP status	Example
400	Document category doesn't match the file
401	Authentication required or token invalid
403	Account lacks the required permission
413	File exceeds 10 MB
415	Unsupported file type
422	Document is unreadable or AI output is invalid
429	Account or IP rate limit reached
500	Unexpected internal error
502 / 503	AI provider or dependency temporarily unavailable

> Do not return raw stack traces, database details, provider secrets or sensitive document text.

6. Database design

The database stores only information needed to operate the service. It does **not** store uploaded medical documents, extracted text or completed explanations.

Tables

`profiles`

Column	Type	Purpose
`id`	UUID, primary key	Same identity as the Supabase Auth user
`role`	Text	`user` or `admin`
`created_at`	Timestamp	Account creation time

`usage_records`

Column	Type	Purpose
`id`	UUID, primary key	Usage record identifier
`user_id`	UUID, foreign key	Account that used the service
`quota_date`	Date	Usage day in Europe/Prague
`attempts`	Integer	Number of counted attempts
`created_at`	Timestamp	Record creation time
`updated_at`	Timestamp	Last update

Add a unique constraint on `(user_id, quota_date)` so concurrent requests cannot create duplicate daily usage records.

`feedback`

Column	Type	Purpose
`id`	UUID, primary key	Feedback identifier
`user_id`	UUID, foreign key	Feedback submitter
`rating`	Small integer, nullable	Optional positive/negative rating
`category`	Text, nullable	Incorrect info, missing info, unsafe advice or technical issue
`comment`	Text, nullable	Optional user comment
`created_at`	Timestamp	Submission time

Do not automatically attach medical files, extracted text or full explanations to feedback.

Schema management

Use one manually maintained `schema.sql` file, as decided. Record schema changes in Git and review the effects before applying them to an existing database.

The backend should use SQLAlchemy with bounded connection pooling and properly scoped database sessions. Retry transient database failures only a limited number of times.

> **Important:** PostgreSQL access policies and server-side authorization must prevent users from reading or modifying other users' records. Never expose a database service-role key to the frontend.

7. Document processing pipeline

Process the document in this order:

1. Verify authentication and enforce request limits.
2. Validate the extension, actual file signature, file size and selected category.
3. Reject password-protected PDFs without requesting the password.
4. Save the upload to a restricted temporary directory with a generated filename.
5. Extract digital PDF text page by page. For image-based documents, use an appropriate vision-based extraction path.
6. Normalize obvious extraction noise without changing medical values or meaning.
7. Check whether the document matches the selected category and is sufficiently readable.
8. Build a source-grounded representation of the extracted text, preserving page numbers and section boundaries.
9. Send the relevant passages to Gemini with strict instructions and the appropriate output schema.

10 Validate the response structure, source references and extracted medical values.

11 Return the explanation and source references to the frontend.

12 Delete the original file and other temporary processing artifacts.

If the server crashes, a cleanup routine must remove abandoned temporary files. Connection interruption alone must not consume a quota attempt if processing never started.

Minimal retrieval and source grounding

For this MVP, retrieval means selecting relevant passages from the uploaded document and supplying those passages to Gemini. We do not need a separate vector database or external medical knowledge base at the start.

For example, if a report contains a cholesterol result on page 2, the explanation should refer to that page and use the exact extracted value and unit.

If the document doesn't contain enough evidence to support a statement, the system should say so rather than fill the gap with a guess.

8. Structured AI outputs

Use Pydantic schemas for the three report types. Reject malformed output rather than silently filling in missing fields.

Laboratory report

- Overall summary
- Important findings
- Values, units and reference ranges
- Plain-language explanation
- Unclear or missing information
- Questions to ask a doctor
- Source references and page numbers
- Safety warnings, when supported

Prescription

- Prescription summary
- Medication names as written
- Instructions explicitly present in the document
- Unclear or missing dosage, frequency or duration
- Questions for a doctor or pharmacist
- Source references and page numbers
- Relevant safety warnings

Never infer missing medication instructions.

Medical report

- Overall summary
- Important findings
- Plain-language explanation
- General healthy lifestyle suggestions, when appropriate
- Unclear or missing information
- Questions to ask a doctor
- Source references and page numbers

- Safety warnings, when supported

Lifestyle suggestions must be separated from facts contained in the report.

Validation rules

- Every displayed medical value must match the extracted source.
- Preserve original units and reference ranges.
- Every source citation must point to a valid extracted page or section.
- Don't invent medication details, diagnoses or missing findings.
- Reject malformed AI output and let the user retry.
- Use one automatic retry for eligible transient provider failures, but not for invalid structured output.
- A valid JSON response is not proof of medical correctness.

> Gemini may help identify concerning findings, but it must not be the sole authority for urgent medical warnings. Do not tell users a report is safe simply because no warning was generated.

9. Authentication, privacy and security

Authentication

- Supabase Auth for signup, sign-in and session management.
- Email verification before the first upload.
- Google sign-in if configured with proper token verification.
- Password reset by email.
- Five-minute, one-time reset OTPs where the chosen authentication flow supports that configuration.
- Rate limits for signup, login, reset requests and upload attempts.
- Explicit backend authorization for admin-only functions.

Use Gmail SMTP only after verifying its current sending limits and suitability for the chosen authentication flow.

Secrets

Keep all secrets in environment variables:

```
DATABASE_URL=
GEMINI_API_KEY=
SUPABASE_URL=
SUPABASE_JWT_SECRET_OR_VERIFICATION_CONFIG=
SMTP_HOST=
SMTP_USERNAME=
SMTP_PASSWORD=
FRONTEND_ORIGIN=
```

Use the verification method recommended by Supabase for the configured token type. Do not assume every Supabase project uses the same JWT verification configuration.

Commit `.env.example` with placeholders, never actual credentials. Add `.env`, temporary uploads and generated private artifacts to `.gitignore`.

Privacy

- No persistent medical-document storage.
- No saved analysis history.
- No medical content in logs.
- Minimal structured logs: request IDs, error categories, timings and sanitized service failures.
- Temporary files deleted after success and permanent failure.

- Cleanup of abandoned files after crashes.
- No automatic attachment of documents to feedback.
- HTTPS in deployed environments.
- Clear privacy notice describing what is sent to Gemini and the applicable provider's data-handling terms.

> Before using real medical documents, review Gemini API data-handling, retention and training terms for the specific account and API configuration. During development, use synthetic or non-sensitive sample documents.

10. Quota and cost controls

> The initial rule is **five processing attempts per account per day**, resetting at midnight in the Europe/Prague timezone.

- Enforce the quota on the backend, never only in the frontend.
- Show remaining attempts and the next reset time.
- Count an attempt when processing genuinely starts.
- Don't unfairly charge users for server-side resource exhaustion or failures before processing starts.
- Prevent concurrent requests from bypassing the quota using a database transaction or atomic update.
- Apply rate limits by account and IP.
- Enforce the 10 MB file limit and reasonable processing timeouts.
- Stop calling Gemini when its quota is exhausted.
- Allow a “Try again” button for temporary failures when appropriate.
- Use bounded retries for transient failures only.

Account deletion through support was the initial MVP choice, but it needs a clear privacy and retention policy. A support workflow must not be used to avoid applicable data-deletion obligations.

11. Frontend design

Public landing page

Explain what Dr. Decode AI does, supported documents, privacy practices and limitations. Include signup and sign-in.

Main application page

Show:

- Three document-type options
- One-file upload area
- Supported formats and 10 MB limit
- Short disclaimer
- Privacy note
- Attempts remaining and reset time

Require sign-in and verified email before upload.

Analysis screen

Use one loading screen with an animated indicator and illustrative messages. Do not show a fake progress percentage or claim that a particular processing stage is complete when the backend cannot verify it.

Allow cancellation. Stop work where possible and clean up temporary files. Handle provider errors clearly.

Results screen

On desktop, show the original document preview alongside the explanation. On mobile, stack the two vertically.

Include:

- Structured explanation
- Source references and page navigation where reliable
- Safety warnings
- Medical disclaimer
- Download explanation PDF
- Feedback controls
- “Analyze another document” button

The new analysis starts fresh, clears the previous result and resets the selected document category.

12. Admin and monitoring

Build a minimal admin view only after the main analysis workflow works.

It may show:

- Usage totals
- Provider failures and error categories
- Response times
- System health
- Sanitized feedback

It must not show medical files, extracted text or full explanations. Admin access must be enforced by the backend, not merely by hiding frontend controls.

No automated monthly API-budget alert system for the MVP. Monitor usage manually.

13. Error handling

Situation	Expected behavior
Unsupported document	Reject with a clear message
Wrong document category	Ask the user to select the correct category
Unreadable pages	Explain what can be read and flag the rest
Missing dosage	Mark as unclear; never guess
Invalid AI structure	Reject output and offer retry
Temporary Gemini failure	One eligible automatic retry, then offer manual retry
Gemini quota exhausted	Stop requests and explain the temporary limitation
Database outage	Bounded recovery attempts, then a clear error
User cancels	Stop processing where possible and clean temporary files
Browser disconnects	Clean up when work terminates; permit a fresh upload

Situation	Expected behavior
Unexpected backend error	Return a generic message and log sanitized details

14. Development and testing strategy

Build and test locally first. Use Postman to verify the backend before investing time in the React interface.

Manual testing is the chosen MVP approach. Keep the code modular so automated tests can be introduced later.

High-risk manual cases to cover include:

- Valid PDF, scanned PDF, JPG, PNG and HEIC.
- File exceeding 10 MB.
- Wrong category and unsupported documents.
- Password-protected and partially unreadable PDFs.
- Missing dosage or unclear medication instructions.
- Lab values with unusual units or absent reference ranges.
- Invalid Gemini output.
- Gemini timeout and exhausted quota.
- Database connection failure.
- Concurrent uploads and daily quota enforcement.
- Unauthorized access to another account's data.
- Cancellation, temporary-file cleanup and interrupted requests.
- Mobile and desktop result layouts.

15. Build roadmap

Phase 1 — Project foundation

Create the repository, Python environment, FastAPI app, configuration system, database connection and health endpoint.

Deliverable: FastAPI runs locally and `/health` works.

Phase 2 — Database and authentication

Configure PostgreSQL/Supabase, create `schema.sql`, connect SQLAlchemy, integrate Supabase Auth and verify access tokens.

Deliverable: A verified user can access protected endpoints.

Phase 3 — Upload and extraction

Implement file validation, temporary storage, PDF text extraction, image processing, category mismatch detection and cleanup.

Deliverable: A valid upload produces structured extracted text with page references.

Phase 4 — Gemini integration

Implement separate prompts and Pydantic schemas for prescriptions, lab reports and medical reports. Add source grounding, validation and bounded transient-error handling.

Deliverable: A valid document produces a structured, source-linked explanation.

Phase 5 — Quota and security

Implement daily quota accounting, rate limits, privacy-safe logging, error handling and cancellation behavior.

Deliverable: The backend enforces the main security and usage rules.

Phase 6 — API integration

Complete the analysis, quota, feedback and health endpoints. Test success and failure cases in Postman.

Deliverable: The full backend workflow works without a frontend.

Phase 7 — Frontend

Build the landing page, authentication screens, document selection, upload flow, loading state, result view, PDF download and feedback.

Deliverable: A user can complete the full workflow in a browser.

Phase 8 — Final validation and deployment

Test locally, inspect privacy behavior, verify cleanup, document environment variables, configure free-tier hosting and HTTPS, and deploy only after reviewing provider limits and terms.

Deliverable: A working portfolio-ready MVP with a clear README and reproducible setup instructions.

16. Definition of done

The MVP is ready when a verified user can upload a supported medical document, receive a structured English explanation grounded in its contents, inspect the source references, download the explanation, and submit feedback.

It must also reject invalid uploads, protect user accounts, enforce quotas, handle AI/database failures safely, and delete uploaded medical files after processing.

No medical content or completed explanations should be retained as application history.

17. Final architectural decisions

Status	Decisions
Locked	Python, FastAPI, PostgreSQL, SQLAlchemy, Gemini, Supabase Auth, React + TypeScript, Cloudflare Pages for frontend hosting, temporary backend file storage, synchronous processing, basic rate limiting, strict output schemas, minimal structured logging, manual SQL schema management, English-only input, and original laboratory units preserved.
Deferred	Backend hosting provider, exact free-tier deployment configuration, database provider setup details, and whether additional infrastructure is needed after local testing.
Not needed initially	Multi-agent architecture, Redis, background workers, a dedicated vector database, persistent document storage, chat history and a complex analytics platform.

The guiding principle is simple: **build the smallest safe, complete version first, then expand based on real needs.**